



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

Nanyang Business School

WEEK 5

The Automation Development Lifecycle



THE WHOLE LIFE CYCLE

Eight phases, end to end

Colour bands group the phases: discover, design, build, and operate.

1

Process Identification

Find the right processes to automate

2

Assessment & Redesign

Map as-is, design to-be

3

Tool & Vendor Selection

Choose the platform and partner

4

Solution Design

Document the target automation

5

Development

Build the bot to the design

6

Testing

Unit, system, and acceptance testing

7

Deployment & Hypercare

Go live with close support

8

Maintenance & CoE

Run, monitor, and scale

Process Identification



INPUTS

- Business pain points and volume data
- List of candidate processes
- Strategy / ROI targets

KEY ACTIVITIES

- 1 Gather candidate processes across departments
- 2 Score each against good-fit criteria
- 3 Check that standardised digital inputs exist
- 4 Confirm stakeholder buy-in and ROI

ARTEFACTS PRODUCED

- Prioritized pipeline
- Feasibility scorecard

Exit gate: A shortlisted process with a positive business case

Owner: Business + Auto. Lead

PHASE 1 • WORKED EXAMPLE

Process Identification

Scoring finance processes against the good-fit criteria

Candidate process	High volume	Rule-based	Digital input	Stable process	Verdict
Accounts Payable (invoices)	✓	✓	✓	✓	Strong fit
Accounts Receivable	✓	✓	✓	✓	Strong fit
Payroll processing	✓	✓	✓	✓	Strong fit
Financial reporting	✗	✓	✗	✓	Partial
Budgeting & forecasting	✗	✗	✗	✗	Poor fit

Artefact: feasibility scorecard. AP invoice processing scores on every criterion, so it becomes the first project. These are process families, scored on their dominant character; a real assessment scores a specific process such as cash application. Financial reporting inputs are largely digital — its difficulty is judgement and changing requirements.

Assessment & Redesign



INPUTS

Chosen process

Access to systems and SMEs

KEY ACTIVITIES

- 1 Document the as-is process step by step
- 2 Note the system/document at each step
- 3 Mark automatable vs. manual steps
- 4 Redesign the to-be so inputs are bot-ready

ARTEFACTS PRODUCED

- As-is process map
- To-be redesign
- Automatability assessment

Exit gate: A to-be design that is technically automatable

Owner: Business Analyst + SME

PHASE 2 • WORKED EXAMPLE

Assessment & Redesign

As-is invoice steps, and which can be automated

Task	System / document	Automatable?
Create purchase order	SAP	✗
Send PO to vendor	Email, PDF	✗
Create goods receipt	SAP	✗
Receive invoice from vendor	Email, PDF	✓
Match invoice to GR and PO	Manual visual inspection	✓
Input invoice details	SAP	✓
Make payment to vendor	SAP, payment system	✓

To-be redesign: vendor emails must carry set keywords, and the PDF must contain all required fields — so the bot can pick up and read them reliably. ✗ means out of scope for this AP invoice bot, not technically impossible — PO and goods receipt creation are routinely automated in SAP by batch job, API or EDI. Payment execution is automatable but is deferred beyond this release — it needs its own approval and segregation-of-duties controls.

Why the as-is comes first

Three things you cannot do without it

A baseline you can defend

Every benefit you claim later is a difference between two numbers. If you never measured the first one, the saving is an assertion, and finance will treat it as one.

A map of where the cost sits

Cost is rarely where people assume. It hides in rework, in waiting, and in the workarounds nobody documented. You find those by observing and tracing the work, not by interview alone.

A definition of what must still be true

The to-be still has to do everything the process did, including the parts that only happen when something goes wrong. The as-is is where that list comes from.

The as-is is not documentation for its own sake. It is the evidence base for the business case, the design and the test plan — all three come out of it.

How to capture the as-is

No single method gives you the whole picture; each one misses something specific

Method	What it is good for	What it misses
Interview the people who do the work	Intent, judgement, local rules	People describe the official process, not what they do
Watch the work being done	Workarounds and re-keying	Rare cases, and behaviour altered by being watched
Pull the numbers from the systems	Volumes, timings, error rates	Everything done outside a system
Sample the real documents	How much the inputs really vary	Why the variation exists
Run a workshop with the whole chain	Handoffs, and who assumes who is checking	Detail — people compress their own steps

What a complete as-is contains

Six things, and the three that are almost always left out

The as-is checklist

- ✓ Every step, and who performs it
- ✓ System or document at each step
- ✓ Time taken — and time waiting
- ✓ Volumes, including the peak
- ✓ Every decision point and its rule
- ✓ What happens when the step fails

The three usually missing

Waiting that belongs to nobody. A queue has no owner, so it appears on no map — yet it is often most of the elapsed time.

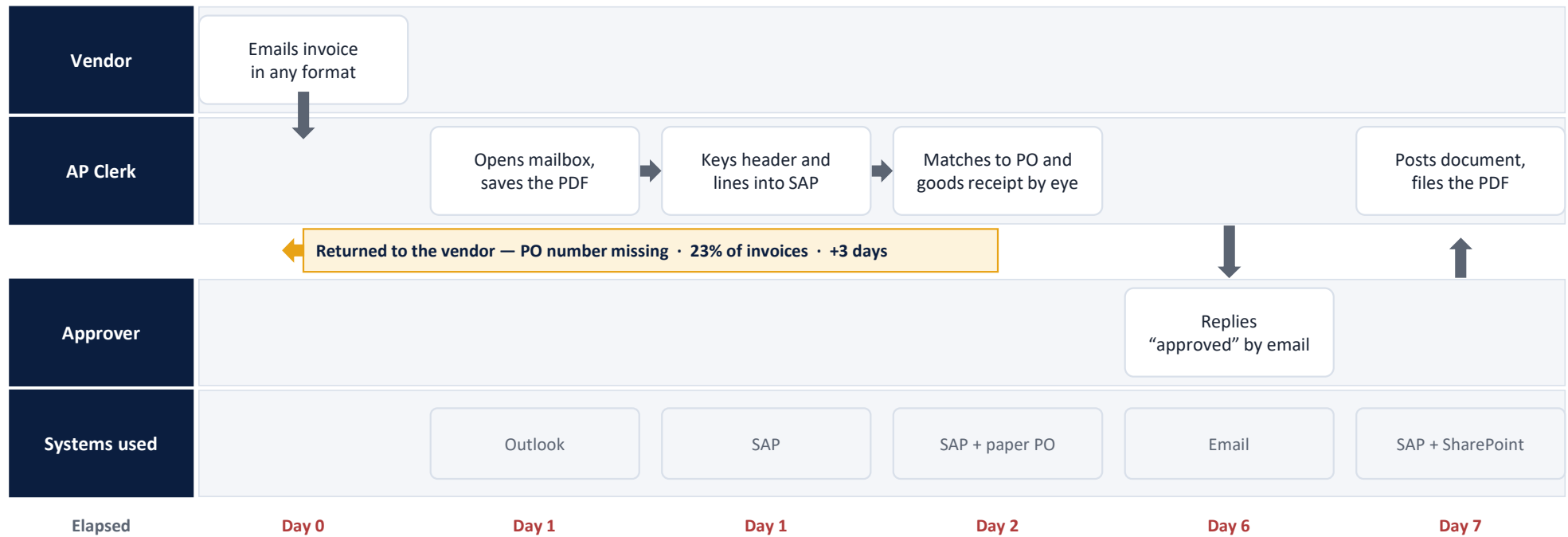
The failure path. Maps are drawn for the case that works. The rework loop is where the cost is, and it is drawn last or not at all.

The undocumented rules. Every team applies rules that are not in any policy. Automate the process without them and it breaks in week one.

PHASE 2 · AS-IS AND TO-BE · WORKED EXAMPLE

As-is swimlane: supplier invoice, today

One lane per participant. Read left to right; the delays are between the boxes, not inside them



Reading the as-is: where the cost hides

Six patterns worth looking for, and what each one is telling you

What you see on the map	What it usually means	What to measure
A loop back to an earlier step	Something was not checked at the point it was created	Share of work that loops, and the delay it adds
A long gap with nobody working	The work is waiting for a person, a batch or a system	Elapsed time against touch time
A handoff between departments	Each side may assume the other is checking	What is actually verified, and by whom
The same data entered twice	Two systems that do not talk, bridged by a person	Keystrokes duplicated, and errors introduced
A spreadsheet beside the system	The system does not do something the work needs	What the spreadsheet holds that the system cannot
Work arriving in batches	A schedule somebody chose, not a constraint	Days lost to the batch window

From as-is to to-be: four moves, in order

Work down the list. Only what survives all four is a candidate for automation

1

Eliminate

Does this step need to happen at all? Checks that duplicate another check, reports nobody reads, approvals below any material threshold.

2

Standardise

Can the input be made consistent at source? A form instead of an email, a fixed format instead of free text, one channel instead of three.

3

Automate

Is what remains rule-based, high volume and stable? Only now does the question of technology arise, and only for the steps that qualify.

4

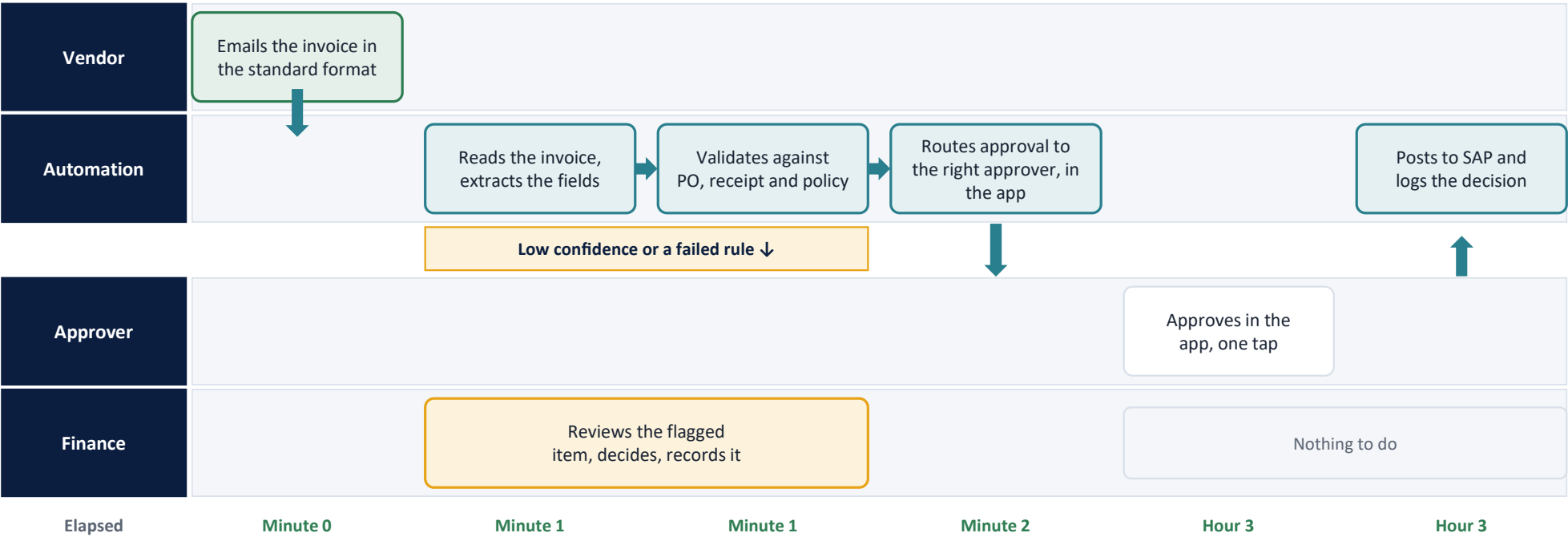
Keep with a person

What needs judgement, accountability or a conversation? Decide this deliberately rather than by default.

The order is the lesson. Automating a step you should have eliminated is the most expensive mistake available in this phase — you pay to build it, then pay to maintain it.

To-be swimlane: the same invoice, redesigned

Same six columns, same participants — what changed is who does each step and how long the gaps are



What changed between the two maps

Every difference traces back to one of the four moves — not to the technology

What changed	Which move	What it bought
Invoices arrive by email in one standard format	Standardise	Removes the source of the three-day return loop
The separate save-and-file step disappears	Eliminate	The document is stored once, at capture
Matching and keying into SAP are done by the automation	Automate	About eight minutes of clerical work per invoice
Approval is routed by amount to the mobile app	Automate	Four days of waiting becomes about three hours
Uncertain reads and failed rules go to a named person	Keep with a person	Errors caught before posting, not after
Every decision is logged with its rule, inputs and time	Automate	The audit trail exists by design

Two of the six changes required no automation at all. Controls are not a fifth move — they are designed into whichever move creates them.

Deciding what a machine does and what a person does

The boundary is a design decision. Made by default, it is made badly

Automate the step when

- The rule can be written down without using the word “usually”
- The input arrives in a form a machine can read
- The volume justifies the build and the upkeep
- The step is stable — it will not be redesigned next quarter
- Getting it wrong is detectable and reversible

Keep a person on the step when

- The decision needs judgement about a specific case
- Somebody has to be accountable by name for the outcome
- The case is too rare or variable to justify a reliable rule
- Accountability for a costly, hard-to-reverse call must rest with a person
- A control requires sign-off by a named human

Design for the exception, not the happy path

The straight-through case takes an afternoon. Everything else is the project

What goes wrong	How the design should notice	Where it should go
The input cannot be read	A confidence score below a threshold you set deliberately	A named person, with the document shown
A rule is not satisfied	The rule fails at the point of submission, not after posting	Back to the originator, with the reason
Two records look identical	A duplicate check performed before acceptance	Blocked; earlier reference shown
A system is unavailable	A retry policy with a limit, then a clean stop	A queue, in its original state — never half-completed
The case needs judgement	The rules do not settle it and the design knows that	A named person, for review and confirmation
The volume spikes	Queue depth and age against a service level	Capacity added, or work shed in a planned order

Controls and evidence in the to-be

Automating a control is not the same as satisfying it

Preventive

Stops the error occurring. Validation at submission; approval routed to the correct level, and separated from whoever posts the entry.

Detective

Finds what got through. A duplicate re-check before payment; a reconciliation between what was submitted and what was posted.

Corrective

Puts it right. An exception queue with an owner; a reversal procedure; a route back into the process for the case that failed.

What the auditor will ask for

Not that the control exists, but that you can show it operating on a specific transaction on a specific day.

Which rule was applied, what the inputs were, what the answer was, and when.

Who approved it — and evidence that it was not the same identity that recorded it.

A record that cannot be edited afterwards without leaving a trace.

Before you commit to build

Eight checks. A gap in any of them is cheaper to close now than in month four

1 Redesigned, not just documented

2 Inputs standardised at source

3 Every rule written down and owned

4 Exception route designed and staffed

5 Machine/person boundary chosen deliberately

6 Controls owned by the business, reviewed with risk or audit

7 Benefit stated as cash and as capacity

8 Scope fixed, deferrals listed

How to-be designs fail

Five failures, each traceable to a decision taken — or avoided — in this phase

The failure	What actually happened	What would have prevented it
The robot automated the mess	The as-is was mapped and then built	Treating the to-be as a redesign, not a transcription
It broke in the first week	Inputs were never standardised	Sampling real inputs and fixing the variation at source
Only the happy path worked	Exceptions were left for later, and later arrived on day one	Designing the exception route alongside the main route
Nobody could explain a decision	The automation was right, but there was no record of why	Specifying the audit trail as part of the design
The expected saving never materialised	Hours were released but never converted to cash or redeployed	Separating cash benefit from capacity released

Tool & Vendor Selection



INPUTS

- To-be design and scope
- Non-functional needs (volume, security)
- Budget

KEY ACTIVITIES

- 1 Compare platforms on capability and cost
- 2 Weigh attended vs. unattended, cloud vs. on-prem
- 3 Assess vendor support and training
- 4 Run a proof of concept before committing

ARTEFACTS PRODUCED

- Tool comparison matrix
- PoC results
- Vendor decision

Exit gate: A selected platform validated by a proof of concept

Owner: IT + Procurement

Tool & Vendor Selection

Comparing platform types for the AP use case (illustrative)

Platform type	Ease of use	Licensing cost	Unattended	Support
Vendor A	Medium	High	Strong	Vendor + partner
Vendor B	High	Medium	Good	Vendor
Open-source	Medium	Free (\$\$0)	Limited	Community

How to choose

Match the tool to the need, not the hype. Unattended, high-volume AP processing favours a platform with strong scheduling and support; a small pilot might start on a low-cost or open-source tool. Whatever the shortlist, run a proof of concept on real invoices before you buy — a demo is not evidence.

Ratings are illustrative teaching values, not a product endorsement. Open-source licensing is free; its cost sits in hosting, support and integration.

What the platform actually costs

Three-year total cost of ownership for one unattended process — illustrative figures

Cost line	Year 1	Year 2	Year 3	What drives it
Platform and tenant licence	S\$18,000	S\$18,000	S\$18,000	Fixed; independent of volume
Unattended robot × 1	S\$10,000	S\$10,000	S\$10,000	One per concurrent execution, not per process
Document reading, ~104k pages	S\$10,400	S\$10,900	S\$11,500	Consumption; grows with volume
Build effort, 60 days	S\$48,000	—	—	One-off; the largest single line
Testing and UAT support, 15 days	S\$12,000	—	—	One-off
Infrastructure and monitoring	S\$15,000	S\$15,000	S\$15,000	Hosting, logging, alerting
Support, 0.2 FTE	S\$12,600	S\$12,600	S\$12,600	Someone must own the live bot
Change requests, ~8 a year	—	S\$9,600	S\$9,600	Interfaces and rules move
Total	S\$126,000	S\$76,100	S\$76,700	Three-year TCO: S\$278,800

Cloud or on-premises

The decision is rarely about capability. It is about data, control and how fast you need to start

Criterion	Cloud	On-premises	What actually decides it
Data residency	Region must be offered and contractually held	Data stays in your estate unless an integration sends it out	Whether the regulator accepts a region commitment
Security review effort	Vendor attestations; shorter but not zero	Your controls, your evidence, longer review	How much of the review your team can absorb
Time to first robot	Days, once tenant, network and identity setup is done	Weeks to months — servers, hardening, access	Whether the pilot has to prove itself this quarter
Upgrade control	Vendor upgrades on their cadence	You choose when, and carry the cost	Whether an unplanned upgrade can break a live bot
Cost shape	Operating expense, scales with use	Capital plus fixed running cost	Which budget line the sponsor controls
Scaling and failover	Add capacity on demand	Capacity is what you bought	How spiky the volume really is
Audit evidence	Vendor logs plus your own	Entirely yours to produce	What the auditor will accept as independent

Infrastructure prerequisites

Seven things that block go-live if they are not arranged in Phase 3 rather than Phase 7

Prerequisite	What it means in practice	Who provides it
Named service accounts	One identity per workload, never a person’s login	IT identity team
Credential vault	Credentials in an encrypted vault with rotation and audit	IT security
Least-privilege access	The robot can do its task in each system	System owners
Network reachability	Endpoints allow-listed, both directions	Network team
Three environments	Development, test and production kept apart	IT and the process owner
Runtime hosts	Machine spec, session policy, patching window	Infrastructure team
Log retention and alert routing	Log retention, and where alerts go at 2am	IT operations

Licensing models and how they scale

Four ways vendors charge, and the volume at which each one stops making sense

Model	You pay for	Scales well when	Breaks down when
Per unattended robot	Each concurrent execution slot	Work is steady and can be queued through one robot	Volume is spiky — you buy for the peak
Per attended user	Each person running automation at their desk	Many people each save a little time	Adoption is patchy — you pay for licences nobody opens
Consumption	Pages read, API calls, transactions	Volume is uncertain, or growing from a small base	Volume is large — a flat licence is cheaper
Per process	Each automated process, regardless of volume	A few high-volume processes	Many small automations — each carries a floor cost

Model the licence against your own volume curve before you negotiate. An eight-minute process gives 7.5 transactions an hour; on one eight-hour shift over 20 working days that is about 1,200 a month — that number, not the list price, decides how many robots you need.

Solution Design



INPUTS

- Selected tool
- To-be process design
- SME knowledge

KEY ACTIVITIES

- 1 Write the PDD: the business process in detail
- 2 Write the SDD: the technical automation approach
- 3 Define exceptions, logging, and escalation
- 4 Get business and IT sign-off before build

ARTEFACTS PRODUCED

- PDD
- SDD
- Exception matrix

Exit gate: A signed-off PDD and SDD

Owner: Solution Architect

PHASE 4 · WORKED EXAMPLE

Solution Design

Two documents, and the target bot flow they describe

PDD — Process Definition

- Process purpose & scope
- As-is / to-be steps
- Business rules
- Volumes & schedule
- Exceptions (business view)

SDD — Solution Design

- Workflow architecture
- Applications & selectors
- Data & config handling
- Logging & audit trail
- Error handling & retries

Target bot flow

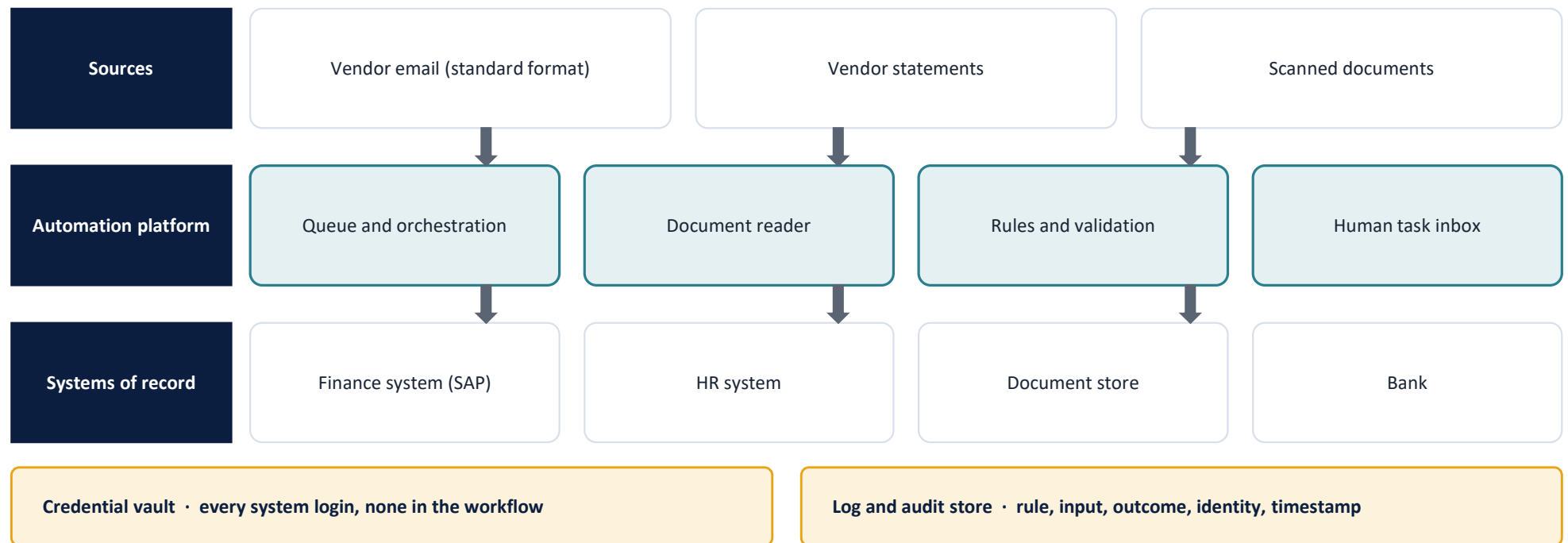
1. Read each email
2. If subject matches the invoice pattern, download PDF
3. Save PDF to “to process”
4. Extract fields from PDF
5. Key data into SAP
6. Update log, move PDF to “done”

Artefacts: PDD + SDD + an exception matrix, all signed off before a single line is built. Step 2 is a pattern match, not a literal search: vendors write INV, Bill, Rechnung, and any mix of case. The SDD specifies the pattern.

PHASE 4 · SOLUTION ARCHITECTURE

Solution architecture at a glance

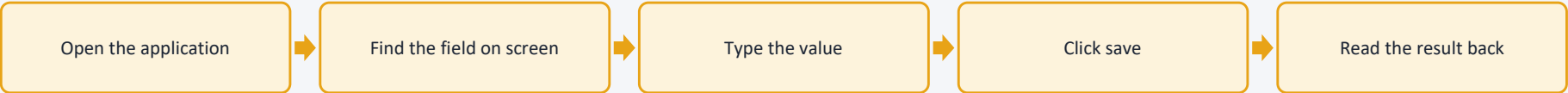
What connects to what — the picture the design document has to describe in words



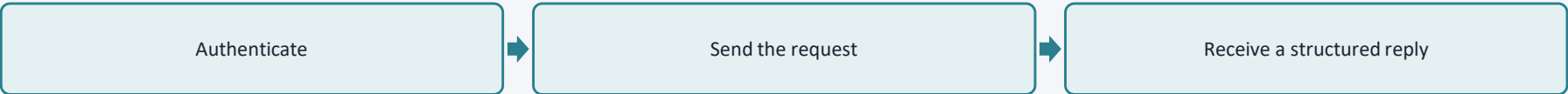
API or user interface: choosing the integration point

The same task, two routes into the same system — and they fail in completely different ways

Interface route



API route



Interface route		API route
Breaks when	The screen layout, a field name or a pop-up changes	Auth, rate limits, outages, or a contract version change
Choose it when	No API is published, or access to it is not granted	One exists, and you are allowed to use it

Development



INPUTS

Signed-off PDD and SDD

Dev environment and access

KEY ACTIVITIES

- 1 Build to the SDD in a dev environment
- 2 Reuse components where possible
- 3 Implement logging and exception handling
- 4 Peer-review and version-control the workflow

ARTEFACTS PRODUCED

- Built bot (dev)
- Code repository
- Dev checklist

Exit gate: A peer-reviewed bot ready for testing

Owner: RPA Developer

PHASE 5 • WORKED EXAMPLE

Development

Turning the design into a built, review-ready bot

Build checklist

- ✓ **Separate dev environment** — not built in production
- ✓ **Modular workflow** — email, PDF, and SAP steps as reusable blocks
- ✓ **Exception handling** — caught at meaningful boundaries, logged and propagated
- ✓ **Clear logging** — counters for emails read, PDFs saved, records keyed
- ✓ **Version control** — workflow committed, changes tracked
- ✓ **Peer review** — a second developer checks before test

Why build discipline matters

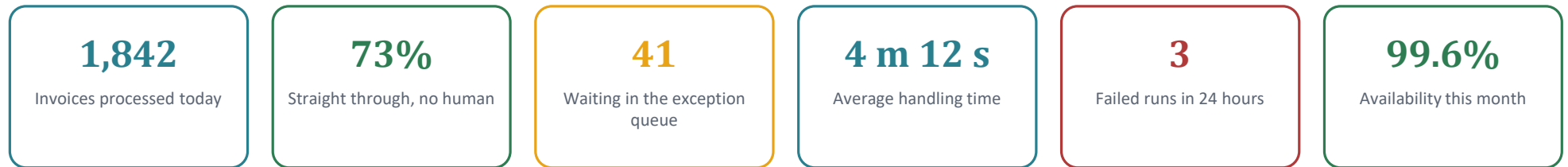
A bot runs unattended, often against systems it does not control. When SAP is slow, an email is malformed, or a PDF is missing a field, the bot must fail safely and say so — rather than silently corrupting data.

That is why logging and exception handling are built in from the start. They are what makes the next phase — testing — able to prove the bot works.

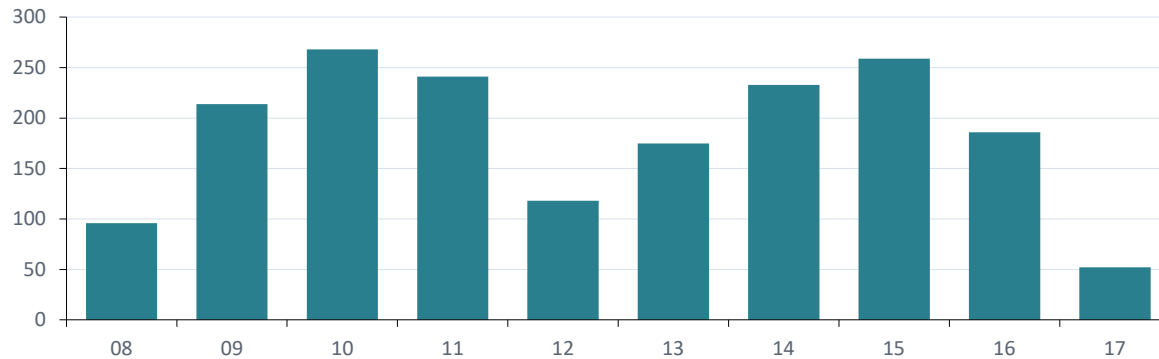
PHASE 5 • WHAT THE RUN LOOKS LIKE

Instrumenting the build: the run dashboard

Built during development, read every day once live — if you cannot see these six numbers, the build is not finished



Invoices processed by hour



What each number is for

Throughput and handling time determine the capacity required.

Straight-through rate tells you whether the design is working.

Queue depth and failures tell you whether today is going wrong.

Availability is what you report against the service level.

Testing



INPUTS

Built bot (dev)

Test data and scenarios

KEY ACTIVITIES

- 1 Unit-test each function, including bad inputs
- 2 System-test the end-to-end flow
- 3 Run UAT with business on real scenarios
- 4 Include negative, robustness and parallel-run testing

ARTEFACTS PRODUCED

- Unit test cases
- System test matrix
- UAT sign-off

Exit gate: UAT signed off by the business

Owner: QA + Business Users

PHASE 6 • WORKED EXAMPLE

Testing

Unit tests first: one function, several inputs

Input (email subject)	What should happen	Expected result
“Invoice #123” + PDF	Read, matches rule, download	PDF saved · counter +1
“INVOICE #456” + PDF (typo intended)	Misspelled — must not be auto-processed	Not processed · routed to manual review
“INVOICE #789”, dup filename	Matches, but the filename clashes	Saved under a unique name · duplicate tested on invoice data

Then system and acceptance testing

System test: run the whole flow across scenarios — single PDF, multiple PDFs, no PDF, wrong subject, duplicate invoice — and check read / save / write at each. **UAT:** business runs real past invoices (historical-data testing), runs in parallel with the manual process, and tries to break it (negative and robustness testing).

Deployment & Hypercare



INPUTS

Tested bot with UAT sign-off

Production environment ready

KEY ACTIVITIES

- 1 Deploy to production with a rollback plan
- 2 Run round-the-clock hypercare monitoring
- 3 Keep a manual plan B running in parallel
- 4 Resolve issues fast, then taper support

ARTEFACTS PRODUCED

- Deployment runbook
- Hypercare log
- Rollback plan

Exit gate: Stable production runs through hypercare

Owner: IT Ops + Auto. Lead

PHASE 7 • WORKED EXAMPLE

Deployment & Hypercare

Going live carefully: deploy, watch closely, keep a fallback

1

Deploy

Move the bot to production with a tested rollback path in case it must be pulled back.

2

Hypercare

Round-the-clock monitoring and fast issue resolution during the first weeks after go-live.

3

Plan B

A manual fallback stands ready, reconciled daily so nothing is processed or paid twice.

4

Taper

Hypercare is temporary — typically two to four weeks — then normal support takes over.

Artefacts: a deployment runbook, a live hypercare log, and a rollback plan you hope not to use.

PHASE 7 • SERVICE LEVELS AND FAILOVER

Service levels: what you are committing to

Six commitments, each measurable from the logs, each with a stated consequence

Commitment	Target	Measured from	If it is breached
Automation availability	99.5% of business hours, monthly	End-to-end run and monitoring logs	Incident raised; root cause within 5 working days
Invoice processed after receipt	95% within 4 business hours	Timestamp in, timestamp posted	Queue reviewed; capacity or threshold adjusted
Straight-through rate	At least 70% monthly	Runs completed with no human touch	Exception mix analysed; rules or reading models adjusted
Exception resolved	90% within 1 business day	Task inbox open and close times	Escalated to the process owner
Failed run restarted	Within 30 minutes in business hours	Alert time to restart time	Support model reviewed
Accuracy of posted fields	99.8% on a monthly sample of 500	Sampled reconciliation	Posting paused for the affected document type

Failover: what happens when it stops

Four situations, four pre-agreed responses — decided now, not at 2am

1

Transient failure

A timeout, a locked record, a momentary network drop

Retry 3 times with backoff, then queue the item

2

Robot host lost

The machine running the automation becomes unavailable

Work reroutes to the standby host within 5 minutes

3

Platform outage

The automation platform itself is down beyond 30 minutes

Fall back to the manual procedure; capacity 200 items a day

4

Upstream system down

The finance or HR system is unavailable

Stop intake, hold the queue, catch up on recovery

Recovery objectives (illustrative, and business-approved in practice): RPO 30 minutes of processing lost, RTO 4 business hours to full service. State both in the runbook, and rehearse the manual fallback before go-live rather than during the first outage.

Maintenance & CoE



INPUTS

Live, stable bot

Support and monitoring in place

KEY ACTIVITIES

- 1 Monitor bot health and handle breakages
- 2 Manage a change pipeline as systems evolve
- 3 Stand up a Centre of Excellence (CoE)
- 4 Feed lessons back into identifying new work

ARTEFACTS PRODUCED

- Monitoring dashboard
- Change log
- CoE charter

Exit gate: A governed, reusable automation capability

Owner: Centre of Excellence

PHASE 8 • WORKED EXAMPLE

Maintenance & CoE

Keeping it alive, and building capability to scale

What breaks a live bot

- SAP or email UI changes and selectors no longer match
- A new invoice format appears that the bot cannot read
- A policy change alters the business rules
- Volumes spike beyond what was tested

Change pipeline

Log the issue → assess impact → fix in dev → re-test → redeploy.
Maintenance is a small version of the whole lifecycle, run again for each change.

Centre of Excellence

A small team that turns one-off projects into a repeatable capability:

- Sets development standards and reusable components
- Governs the pipeline of new automation requests
- Owns monitoring, maintenance, and support
- Tracks benefits and feeds lessons back to Phase 1

Queue depth and alert thresholds

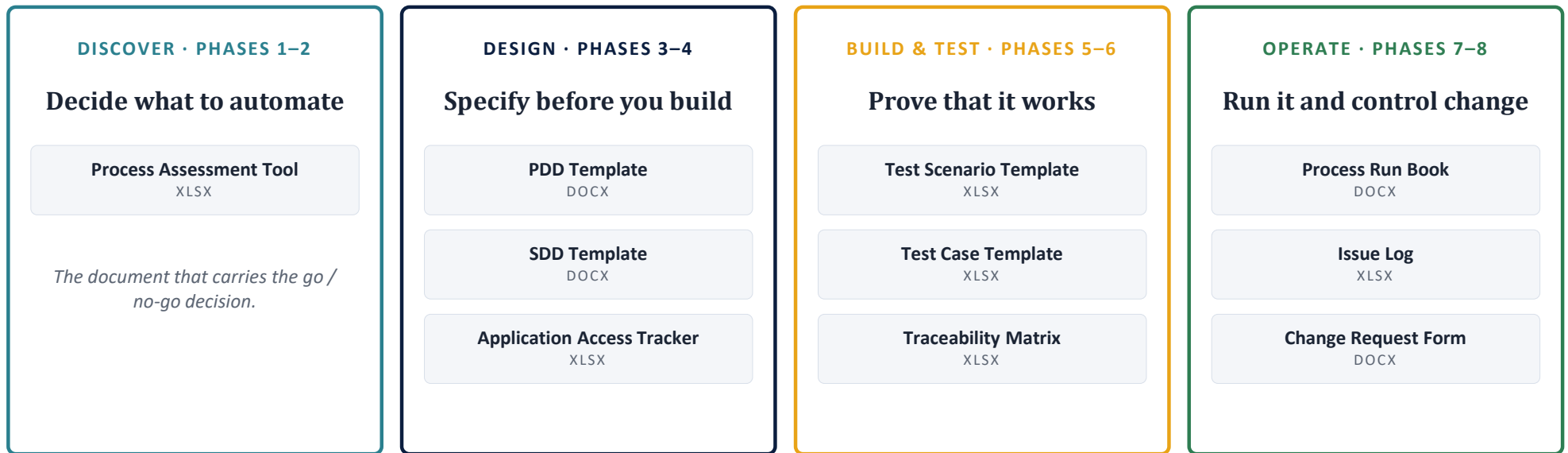
Six signals the Centre of Excellence watches — illustrative thresholds, calibrated to your volume, service levels and staffing

Signal	Normal	Warning	Alert	What it triggers
Items waiting in the queue	Under 50	50 to 150	Over 150	Add a robot for the shift, or throttle intake
Age of the oldest item	Under 2 hours	2 to 6 hours	Over 6 hours	Service level at risk — escalate to the process owner
Exception queue depth	Under 20	20 to 60	Over 60	Staffing gap, or a rule that is failing systematically
Failed runs in 24 hours	0 to 2	3 to 8	Over 8	Stop and diagnose before the backlog compounds
Reads below the confidence threshold	Under 10%	10 to 20%	Over 20%	A new document format has appeared upstream
Robot utilisation	Under 60%	60 to 85%	Over 85%	Capacity trigger — plan the next licence before the peak

ARTEFACTS IN PRACTICE

Ten templates for eight phases

The Automation Business Analysis pack, and the stretch of the lifecycle each document serves



Read it against the lifecycle map: these ten are the documents you reuse on every project. Others the phases name — feasibility scorecard, PoC results, deployment runbook, CoE charter — you write yourself.

Ten templates, one matrix

What each document is, what you fill in, and where it sits in the lifecycle

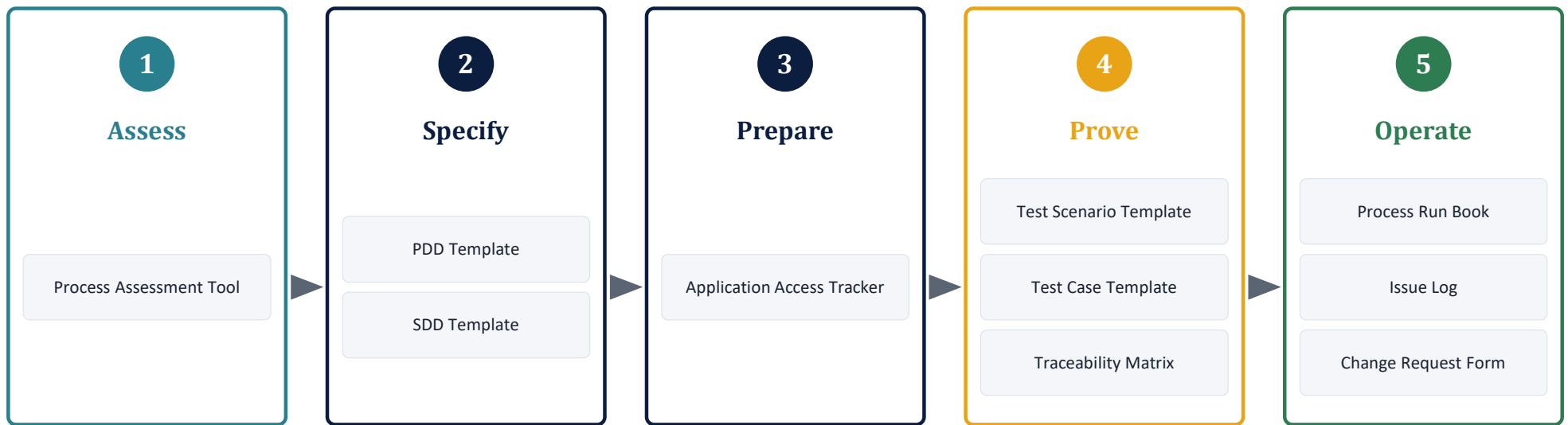
Document	Format	What it is	What you fill in	Phase
Process Assessment Tool	XLSX	Scores a candidate process for fit and benefit	Volumes, rule clarity, systems, exception rate, time per case	1–2
PDD Template	DOCX	The business view, in enough detail to build from	As-is and to-be steps, rules, exceptions, volumes, screenshots	2–4
SDD Template	DOCX	How the robot will actually do it	Workflow structure, components, selectors, config, error handling	4
Application Access Tracker	XLSX	Every application and account the robot needs	System, environment, account, rights, requested and approved dates	3–5
Test Scenario Template	XLSX	The situations worth testing	Scenario, precondition, test data, expected outcome	6
Test Case Template	XLSX	Step-level tests that prove each scenario	Steps, input, expected, actual, pass or fail, tester, date	6
Traceability Matrix	XLSX	Links requirement to design to test	Requirement ID, PDD reference, SDD component, test case ID, result	4–6
Process Run Book	DOCX	How to operate the live robot	Schedule, start and stop, input and output, known errors, escalation	7
Issue Log	XLSX	Every defect and incident in one place	ID, description, severity, raised by, owner, status, resolution	6–8
Change Request Form	DOCX	The controlled route to change a live robot	What changes and why, systems affected, effort, risk, approvals	8

Watch the sequence: the assessment tool justifies the project, the PDD describes the process, the SDD describes the automation. Neither the PDD nor the SDD is signed until both are written.

HOW THE PACK HANGS TOGETHER

One chain, ten documents

Each document is an input to the next — which is why skipping one shows up later



And it loops: maintenance is this same chain run again, smaller. An approved change request re-enters at Specify, and the test cases, matrix and run book all get updated with it. Several documents are written in parallel: the chain shows dependency, not a strict order.

ROLES ACROSS THE LIFECYCLE

RACI: who does what

Responsible does the work · Accountable owns the outcome · Consulted is asked · Informed is told

Role	P1	P2	P3	P4	P5	P6	P7	P8
Business Sponsor	A	I	C	I	I	I	I	I
Process Owner / SME	R	A	C	C	I	A	C	C
Business Analyst	R	R	C	C	I	I	I	C
Solution Architect	C	C	R	A	C	C	C	C
RPA Developer	I	I	I	R	A/R	R	C	R
QA / Tester	I	I	I	C	C	R	C	C
IT / Infrastructure	C	C	A	C	C	C	A	C
Centre of Excellence	I	C	C	C	C	C	R	A

P1 Process Identification
P2 Assessment & Redesign
P3 Tool & Vendor Selection
P4 Solution Design

P5 Development
P6 Testing
P7 Deployment & Hypercare
P8 Maintenance & CoE

One Accountable role per phase. Acceptance in P6 is owned by the business; QA runs and evidences the testing.

Key takeaways

- 1 It is a lifecycle, not a build**

Eight phases, each with inputs, activities, and artefacts. The build is only one of them.
- 2 Artefacts chain the phases**

A pipeline feeds a design, a design feeds the build, the build feeds testing. Skip one and it shows later.
- 3 Gates control quality**

Each phase ends in a go / no-go. A bad-fit process or a failed test stops the line before more is spent.
- 4 Design before you build**

The PDD and SDD are signed off before development. Skipping design sign-off is a major source of avoidable rework.
- 5 Operate, do not just deploy**

Hypercare, maintenance, and a CoE keep the bot alive — and loop lessons back to Phase 1.